

8.6 토픽 모델링— 20 뉴스그룹

1. 토픽 모델링 개념

: 대규모 문서 집합에서 잠재적인 주제를 자동으로 추출하는 기법

- 사람이 직접 문서를 읽어 주제를 파악하는 비효율성 해결

2. 대표적인 토픽 모델링 기법

: LSA (Latent Semantic Analysis), LDA (Latent Dirichlet Allocation)

** Linear Discriminant Analysis(LDA)와 이름만 같고 완전히 다른 알고리즘**

3. 데이터 수집 및 벡터화

-데이터 로딩

fetch_20newsgroups() API 사용

categories 파라미터로 필요한 주제만 필터링

- 벡터화 방식 : CountVectorizer 사용

- 주요 파라미터

max_features = 1000

ngram_range = (1, 2)

- 결과 : 7862개 문서 × 1000개 단어 피쳐 행렬 생성

4. LDA 모델 학습

- 사용 모델 : LatentDirichletAllocation

- 주요 설정

토픽 수: n_components = 8

random_state 설정 → 결과 재현성 확보

- 학습 결과 : components_ 속성 생성

5. components_ 속성 의미

- 의미 : 행(row): 토픽 / 열(column): 단어 피쳐

- 값 해석 : 값이 클수록 해당 단어가 그 토픽의 중심 단어

- 예시 : components_[0, 10] → Topic #0에서 10번째 단어의 연관도

6. 토픽 해석 방법

components_ 값만으로는 해석이 어려움

display_topics() 함수 사용 : 토픽별 연관도 높은 단어 정렬 출력

- 목적 : 각 토픽의 의미 파악

7. 정리 및 한계

- 장점 : 대규모 문서 집합의 전반적인 주제 구조 파악 가능
- 한계 : 토픽 수 설정에 민감, 데이터 품질 및 전처리에 따라 결과 차이 큼
- 결론 : 토픽 모델링은 정답을 찾는 기법이 아니라 탐색적 분석 도구

8-7 문서 군집화

1. 문서 군집화 개념

: 문서 군집화: 텍스트 내용이 유사한 문서들을 자동으로 묶는 기법

- 비지도학습 기반 (라벨 필요 없음)
- 텍스트 분류와 차이점

텍스트 분류: 지도학습, 사전 라벨 필요

문서 군집화: 비지도학습, 유사도 기반 자동 분류

2. 텍스트 피처 벡터화

- 벡터화 방식: TF-IDF
- 사용 객체: TfidfVectorizer
- 주요 설정

tokenizer: LemNormalize (토큰화 + 불용어 제거 + 어근 변환)

ngram_range = (1, 2)

min_df, max_df 설정 → 피처 수 제한

- 결과 : 문서별 TF-IDF 피처 벡터 행렬 생성

3. K-means 기반 문서 군집화

- 사용 알고리즘: KMeans

(1) 군집 개수 = 5

- max_iter = 10000

- 결과 특징

호텔 리뷰 군집 형성

포터블 전자기기(킨들, 아이패드 등) 군집

차량용 내비게이션 군집

자동차 리뷰 군집

- 문제점 : 군집 개수가 많아 과도하게 세분화됨

(2) 군집 개수 = 3

- 군집 결과 개선

Cluster #0: 전자제품 리뷰

Cluster #1: 자동차 리뷰

Cluster #2: 호텔 리뷰

- 전체적으로 명확한 군집 형성

4. 군집별 핵심 단어 추출

- 사용 속성: cluster_centers_

- 의미 : 각 군집 중심과 단어 피쳐 간의 상대적 거리로, 값이 클수록 해당 군집의 핵심 단어

- 배열 형태 : (군집 수, 피쳐 수)

- argsort()로 큰 값의 인덱스 추출 / 인덱스를 통해 단어 이름 매핑

- 사용 함수 : get_cluster_details(), print_cluster_details()

5. 군집별 핵심 키워드 결과

- 전자제품 군집 : screen, battery, battery life / 화면크기, 배터리 수명에 대한 관심 반영

- 자동차 군집 : interior, seat, mileage, comfortable / 실내 인테리어, 연비, 승차감 중심
- 호텔 군집 : room, hotel, service, staff / 객실 상태, 서비스, 직원 응대 중심

8.8 문서 유사도

1. 문서 유사도 개념

- 문서 유사도: 문서와 문서 간 내용이 얼마나 비슷한지 측정
- 일반적으로 코사인 유사도 사용

2. 코사인 유사도

- 벡터 간 크기가 아닌 방향의 유사성 측정
- 두 벡터 사이의 사잇각($\cos \theta$)을 이용
- 값의 범위

1 : 완전히 동일한 방향 (매우 유사)

0 : 서로 무관

-1 : 정반대 방향

3. 코사인 유사도 수식

- 두 벡터 A, B의 내적 : $A \cdot B = \|A\| \times \|B\| \times \cos\theta$
- 코사인 유사도 공식 : $\text{similarity} = (A \cdot B) / (\|A\| \times \|B\|)$
- 의미 : 벡터 내적을 벡터 크기로 정규화한 값

4. 문서 유사도에 코사인 유사도를 쓰는 이유

- 문서 벡터는 대부분 고차원 희소 행렬
- 유클리드 거리 기반 유사도 문제점

: 문서 길이에 민감하며 단어 빈도 수에만 영향 받음

- 코사인 유사도의 장점 : 문서 길이 차이의 영향 적음, 단어 빈도보다는 비율과 방향성에 집중

- 긴 문서에서 특정 단어가 많이 등장했다고 해서 더 관련 있다고 판단 X

5. 코사인 유사도 직접 구현

- NumPy 기반 함수

```
def cos_similarity(v1, v2):  
    dot_product = np.dot(v1, v2)  
    l2_norm = (np.sqrt(sum(np.square(v1))) * np.sqrt(sum(np.square(v2))))  
    return dot_product / l2_norm
```

- 입력 : 두 개의 벡터(array)

6. 간단한 문서 유사도 실습

- 문서 리스트(doc_list)를 TF-IDF로 벡터화
- 결과 : 희소 행렬 → 밀집 행렬로 변환 후 array 사용
- 유사도 결과 예시

문서1 vs 문서2 : 0.402

문서1 vs 문서3 : 0.404

문서2 vs 문서3 : 0.456

7. 사이킷런 cosine_similarity API

- 함수 : sklearn.metrics.pairwise.cosine_similarity
- 특징

희소 행렬 / 밀집 행렬 모두 지원

별도 변환 작업 불필요

- 입력 : 비교 기준/대상 문서 행렬
- 출력 : 코사인 유사도 값 ndarray

8. 코사인 유사도 사용 예

- 첫 번째 문서를 기준으로 비교 : 자기 자신과의 유사도 = 1
- 자기 자신 제외 방법 : feature_vect[1:] 사용
- 쌍(pairwise) 유사도 계산 가능 : 모든 문서 간 유사도 행렬 반환
- 반환 형태 : (문서 수 × 문서 수) ndarray

9. 호텔 리뷰 문서 유사도 측정 절차

- document_df에서 호텔 군집 문서 인덱스 추출
- 기존 TF-IDF 벡터에서 해당 문서 벡터 추출
- 별도의 재벡터화 X
- 코사인 유사도로 문서 간 유사도 계산

10. 유사도 결과 정렬 및 해석

- 유사도 결과를 reshape(-1)로 1차원 변환
- 유사도 높은 순으로 정렬
- 결과 예시

기준 문서 : bathroom_bestwestern_hotel_sfo

가장 유사한 문서 : room_holiday_inn_london

코사인 유사도 ≈ 0.514

8.9 한글 텍스트 처리 — 네이버 영화 평점 감성 분석 요약 정리

1. 감성 분석 개요

- 목표 : 네이버 영화 평점 데이터를 이용한 긍정/부정 감성 분석
- 방식 : 텍스트 분류 문제, 지도학습 기반

2. 한글 NLP 처리의 어려움

(1) 띄어쓰기 문제

: 띄어쓰기 오류 시 의미 왜곡 가능

(2) 조사 처리 문제

: 조사 종류가 매우 다양, 어근 추출 및 제거 어려움

3. KoNLPy 소개

: 파이썬 대표 한글 형태소 분석 패키지

- 형태소 : 의미를 가지는 최소 단위

- 형태소 분석 : 형태소 분리 + 품사 태깅(POS Tagging)

- KoNLPy 특징

기존 C/C++, Java 기반 형태소 엔진을 파이썬 래퍼로 제공

안정성과 신뢰성 확보

4. 데이터 로딩 및 탐색

- 데이터 파일 : ratings_train.txt, ratings_test.txt

- 로딩 방식 : pd.read_csv() / 구분자: \t, 인코딩: cp949

- 데이터 구조

document: 리뷰 텍스트

label: 감성 값 (1: 긍정, 0: 부정)

- 라벨 분포 : 긍정/부정 비율 균등

5. 데이터 전처리

- Null 값 처리 : document 컬럼의 NaN → 공백

- 정규 표현식 처리 : 숫자 제거 (re 사용)

- 학습/테스트 데이터 동일한 전처리 적용

6. 형태소 기반 토큰화

- 사용 분석기 : Twitter(Okt)

- 토큰화 방식 : morphs() 메서드

- 사용자 정의 tokenizer : tw_tokenizer()

- 역할 : 문장 → 형태소 단어 리스트

- 사용 위치 : TfidfVectorizer의 tokenizer 파라미터

7. TF-IDF 피처 벡터화

- 사용 객체 : TfidfVectorizer

- 주요 설정

tokenizer = tw_tokenizer

`ngram_range = (1, 2)`

`min_df = 3`

`max_df = 0.9`

- 특징 : 형태소 단위 + N-gram 반영
- 주의사항 : 실행 시간 김 (약 10분 이상 소요)

8. 감성 분류 모델

- 사용 모델 : Logistic Regression
- 하이퍼파라미터 튜닝 : GridSearchCV 사용 / 파라미터: C
- 최적 결과 : C = 3.5 / 정확도 $\approx$ **0.8593**

9. 테스트 데이터 예측

- 중요 원칙 : 학습에 사용한 TfidfVectorizer 재사용
- 이유 : 학습/테스트 피쳐 차원 동일성 유지
- 절차 : `tfidf_vect.transform(test_df['document'])`, 모델로 예측 수행